



LAB EXERCISES

COURSE – FULL STACK WEB DEVELOPMENT

COURSE CODE- PAMCA506

NAME- PRIYANSHU RATHORE

REGNO- 25MCA1032

Ex: 1	As a web developer you are given the task of designing an online registration form for an event called “MasterChef Chennai” using HTML and JavaScript. The form must obtain the following details and perform validations as specified: • Textbox – name of the contestant (cannot be empty) • Number – age of the contestant (age in range of 20-60 inclusive) • Email – email of the contestant (must be in correct format, if not display a custom error text “Correct mail-id please”) • Textbox – pin code (must be 6 digits, check using regular expression) • Submit button Note: • The above validations must be performed using HTML attributes • When there is invalid data, style respective controls with a box shadow of blue color and default black if valid. Perform this using appropriate CSS pseudo-selectors. • E-mail validation must be performed using JavaScript validation DOM API to display the custom text.
-------	---

Index.html:-

```

<!DOCTYPE html>
<html>
<head>
  <title>MasterChef Chennai Registration</title>

  <style>
    input:valid {
      box-shadow: 0 0 5px black;
    }

    input:invalid {
      box-shadow: 0 0 5px blue;
    }
  </style>
</head>

<body>

  <h2>MasterChef Chennai Registration</h2>

  <form id="regForm">

    <!-- Name -->
    Name:
    <input type="text" required>
    <br><br>

    <!-- Age -->
    Age:
  
```

```

<input type="number" min="20" max="60" required>
<br><br>

<!-- Email -->
Email:
<input type="email" id="email" required>
<br><br>

<!-- Pin Code -->
Pin Code:
<input type="text" pattern="[0-9]{6}" required>
<br><br>

<button type="submit">Submit</button>

</form>

<script>
  const emailField = document.getElementById("email");

  emailField.addEventListener("input", function () {

    if (emailField.validity.typeMismatch) {
      emailField.setCustomValidity("Correct mail-id please");
    } else {
      emailField.setCustomValidity("");
    }

  });
</script>

</body>
</html>

```

OUTPUT:-

MasterChef Chennai Registration

Name:

Age:

Email:

Pin Code:

Ex 2: Develop an online greeting website using jquery. Assume that the webpage has paragraph with attribute called "title", <h1> tag without text, tag without "src"

attribute and a <h5> tag with text “XXXX”. The value of the “title” attribute is “Happy Birthday”. The webpage must have a button called “Greet”. On clicking the “Greet” button perform event handling as follows using jquery:

- Assign the “title” attribute value to the text of <h1> tag.
- Change the styling of background color of the <h1> text to “pink”.
- Set the “src” attribute of tag to “C:/birthday.jpg”.
- Replace the content of <h2> tag with text “XXXX” to “Kevin”.

Index.html:-

```
<!DOCTYPE html>
<html>
<head>
  <title>Greeting Page</title>

  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

  <style>
    body {
      font-family: Arial;
      background-color: blueviolet;
      text-align: center;
    }

    h1 {
      padding: 10px;
    }
  </style>
</head>

<body>

  <p id="para" title="Happy Birthday"></p>
  <h3>click the button to greet </h3>

  <h1></h1>

  <img id="img" width="200">

  <br><br>

  <h2>XXXX</h2>

  <br>

  <button id="btn">Greet</button>

  <script>
```

```

$(document).ready(function () {

    $("#btn").click(function () {

        var msg = $("#para").attr("title");
        $("h1").text(msg);

        $("h1").css("background-color", "pink");

        $("#img").attr("src", "C:/birthday.jpg");

        $("h2").text("Kevin");

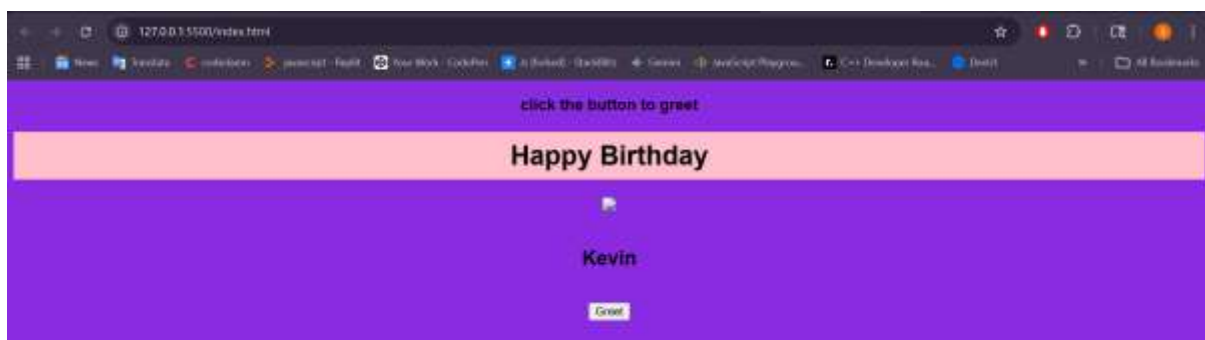
    });

});
</script>

</body>
</html>

```

OUTPUTS:-



Ex 3: Develop a SPA (Single Page Application) for online verification of age eligibility to cast vote during the coming polls using Angular JS. The application consists of 3 components namely AppComponent (parent component) and two child components namely eligible and not-eligible. In the AppComponent obtain the name and age of the voter in textboxes and on clicking the “Verify” button display the voter’s name in

AppComponent in an ordered list. Simultaneously send the voter object (including name and age) as inputs to eligible and not-eligible components. If the age of the voter is greater than or equal to 18, then display the name of the voter in an ordered list in eligible component. If the age of the voter is lesser than 18, then display the name of the voter in an ordered list in not-eligible component. The two child components must be embedded in AppComponent such that AppComponent displays all the names, followed by eligible voter's names and finally not eligible voter's names. Perform the above using inter component communication.

Note:

- For all the three components write both the component class (typescript file) and template (html file).
- Write the appropriate interface to represent a voter.
- Write the steps for app creation and component creation in CLI.
- Assume <app-eligible> and <app-not-eligible> as selectors of eligible and not-eligible components respectively.

App.ts:-

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { EligibleComponent } from './eligible/eligible';
import { NotEligibleComponent } from './not-eligible/not-eligible';
import { Voter } from './voter';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [
    CommonModule,
    FormsModule,
    EligibleComponent,
    NotEligibleComponent
  ],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class AppComponent {

  name: string = "";
  age: number = 0;

  allVoters: Voter[] = [];
  currentVoter?: Voter;

  verifyVoter() {
    this.currentVoter = {
```

```

    name: this.name,
    age: this.age
  };

  this.allVoters.push(this.currentVoter);

  this.name = "";
  this.age = 0;
}
}

```

Main.ts:-

```

import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app';
import { appConfig } from './app/app.config';

bootstrapApplication(AppComponent, appConfig)
  .catch(err => console.error(err));

```

Eligible.ts:-

```

import { Component, Input } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Voter } from '../voter';

@Component({
  selector: 'app-eligible',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './eligible.html',
  styleUrls: ['./eligible.css']
})
export class EligibleComponent {

  @Input() voter?: Voter;
  eligibleList: string[] = [];

  ngOnChanges() {
    if (this.voter && this.voter.age >= 18) {
      this.eligibleList.push(this.voter.name);
    }
  }
}

```

Not-eligible.ts:-

```

import { Component, Input } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Voter } from '../voter';

@Component({
  selector: 'app-not-eligible',

```

```

standalone: true,
imports: [CommonModule],
templateUrl: './not-eligible.html',
styleUrl: './not-eligible.css'
})
export class NotEligibleComponent {

  @Input() voter?: Voter;
  notEligibleList: string[] = [];

  ngOnChanges() {
    if (this.voter && this.voter.age < 18) {
      this.notEligibleList.push(this.voter.name);
    }
  }
}

```

Voter.ts:-

```

export interface Voter {
  name: string;
  age: number;
}

```

App.html:-

```

<h2>Voter Verification</h2>

<label>Name:</label>
<input type="text" [(ngModel)]="name">

<br><br>

<label>Age:</label>
<input type="number" [(ngModel)]="age">

<br><br>

<button (click)="verifyVoter()">Verify</button>

<hr>

<h3>All Voters</h3>
<ol>
  <li *ngFor="let v of allVoters">
    {{ v.name }}
  </li>
</ol>

<hr>

<app-eligible [voter]="currentVoter"></app-eligible>
<app-not-eligible [voter]="currentVoter"></app-not-eligible>

```


Eligible.html:-

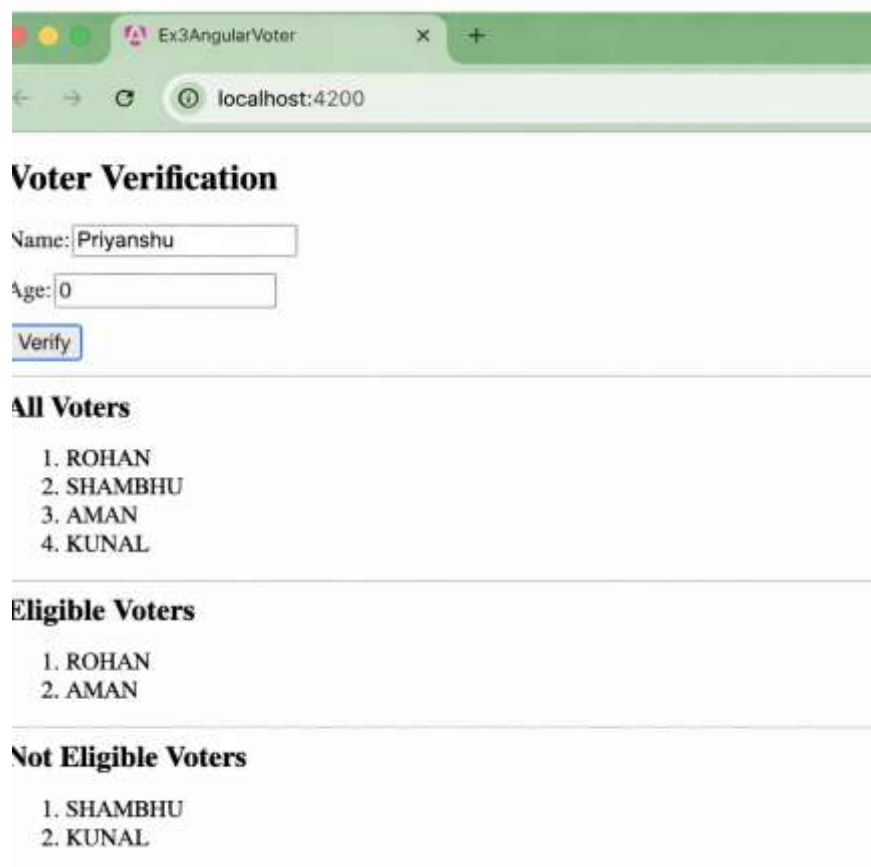
```
<h3>Eligible Voters</h3>

<ol>
  <li *ngFor="let name of eligibleList">
    {{ name }}
  </li>
</ol>
```

Not-eligible.html:-

```
<h3>Not Eligible Voters</h3>

<ol>
  <li *ngFor="let name of notEligibleList">
    {{ name }}
  </li>
</ol>
```

OUTPUTS:-


Voter Verification

Name:

Age:

All Voters

1. ROHAN
2. SHAMBHU
3. AMAN
4. KUNAL

Eligible Voters

1. ROHAN
2. AMAN

Not Eligible Voters

1. SHAMBHU
2. KUNAL

Ex 4:	Eric's Pizzeria is a new Pizza shop in town. They have approached you to develop a SPA (Single Page Application) using Angular JS for online ordering. The web app consists of 3 components. The parent component (AppComponent) must display the cost of an order where it is initially set to zero. Create a child component to place
-------	---

order, namely “MenuComponent” that has buttons (+) to add one vegetarian pizza and (-) to remove one vegetarian pizza. Similarly, it has (+) and (-) buttons for adding and removing one non-vegetarian pizza. MenuComponent fetches initial cost as input from AppComponent. On clicking the (+) and (-) buttons compute the price of the order using event binding in MenuComponent. Assume the cost of one vegetarian pizza is Rs. 40 and non-vegetarian pizza is Rs. 80. Update the computed cost in AppComponent by sending cost as output from MenuComponent. Create another child component called “TotalComponent”. “TotalComponent” must get cost as input from AppComponent. TotalComponent must have a button called “TotalCost” which on clicking will deduct Rs.10 if the cost is greater than or equal to Rs. 500 and retain the same amount if not. A GST of Rs. 20 must be added to cost in both cases. Update the final cost of the order in AppComponent by sending the final cost as output from “TotalComponent”. Perform the above using inter component communication.

Note:

- For all the three components write both the component class (typescript file) and template (html file).
- Write the steps for app creation and component creation in CLI.
- Assume <app-menu> and <app-total> as selectors of MenuComponent and TotalComponent class respectively.

App.ts:-

```
import { Component } from '@angular/core';
import { MenuComponent } from './menu/menu';
import { TotalComponent } from './total/total';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [MenuComponent, TotalComponent],
  templateUrl: './app.html'
})
export class AppComponent {

  cost = 0;
  finalCost = 0;

  updateCost(val: number) {
    this.cost = val;
  }

  updateFinal(val: number) {
    this.finalCost = val;
  }
}
```

App.html:-

```
<h1>Pizza App</h1>

<app-menu
  [cost]="cost"
```

```

    (costChange)="updateCost($event)">
</app-menu>

<hr>

<app-total
  [cost]="cost"
  (finalCostEvent)="updateFinal($event)">
</app-total>

<h2>Final Cost: ₹{{finalCost}}</h2>

```

Menu.ts:-

```

import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-menu',
  standalone: true,
  templateUrl: './menu.html'
})
export class MenuComponent {

  @Input() cost: number = 0;
  @Output() costChange = new EventEmitter<number>();

  addVeg() {
    this.cost += 15;
    this.costChange.emit(this.cost);
  }

  removeVeg() {
    this.cost -= 15;
    this.costChange.emit(this.cost);
  }

  addNonVeg() {
    this.cost += 30;
    this.costChange.emit(this.cost);
  }

  removeNonVeg() {
    this.cost -= 40;
    this.costChange.emit(this.cost);
  }
}

```

Menu.html:-

```

<div class="box">
  <h3>Menu</h3>

  <p>Veg Pizza</p>

```

```

<button (click)="addVeg()">+</button>
<button (click)="removeVeg()">-</button>

<p>Non-Veg Pizza</p>
<button (click)="addNonVeg()">+</button>
<button (click)="removeNonVeg()">-</button>

<p>Cost: ₹{{cost}}</p>
</div>

```

Total.ts:-

```

import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-total',
  standalone: true,
  templateUrl: './total.html'
})
export class TotalComponent {

  @Input() cost: number = 0;
  @Output() finalCostEvent = new EventEmitter<number>();

  calculate() {
    let final = this.cost;

    if (final >= 500) {
      final -= 10;
    }

    final += 20;

    this.finalCostEvent.emit(final);
  }
}

```

Total.html:-

```

<div class="box">
  <h3>Total</h3>

  <button (click)="calculate()">TotalCost</button>

  <p>Cost: ₹{{cost}}</p>
</div>

```

Styles.css:-

```

body {
  font-family: Arial;
  background: linear-gradient(to right, #ff9966, #ff5e62);
}

```

```

text-align: center;
}

.box {
background: white;
padding: 20px;
margin: 20px auto;
width: 250px;
border-radius: 10px;
}

button {
margin: 5px;
padding: 5px 10px;
}

```

Main.ts:-

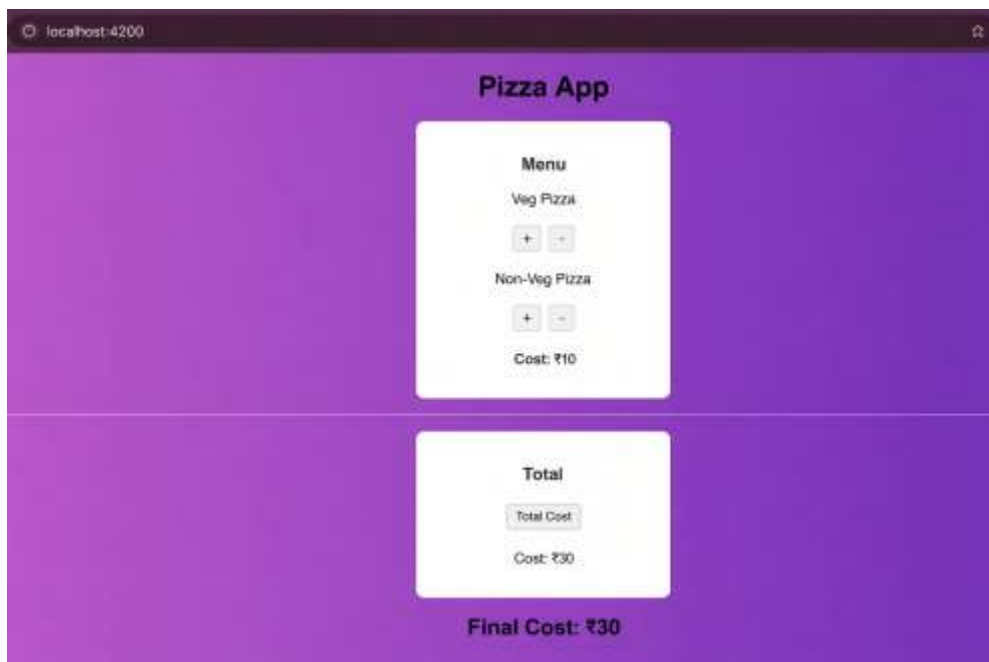
```

import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app';

bootstrapApplication(AppComponent)
  .catch(err => console.error(err));

```

OUTPUTS:-



Ex 5	Using JavaScript and the DOM, create a 3×3 grid of buttons representing seats in a movie theatre.
------	---

- Each button must display its row and column number (e.g., "Row 1, Col 2"). When a user clicks a button to book a seat, the button's color should change from green to red to indicate it is booked. All booked seats must remain red.
- Additionally, calculate the total price of booked seats at ₹250 per seat and display the total dynamically inside an `<h1>` tag using DOM manipulation.

Index.html:-

```
<!DOCTYPE html>
<html>
<head>
  <title>Movie Seat Booking</title>

  <style>
    button {
      width: 100px;
      height: 50px;
      margin: 5px;
      background-color: green;
      color: white;
      border: none;
      cursor: pointer;
    }

    button.booked {
      background-color: red;
      cursor: not-allowed;
    }
  </style>
</head>

<body>

  <h1>Total Amount: ₹0</h1>

  <div id="seats"></div>

  <script src="script.js"></script>
</body>
</html>
```

Script.js:-

```
const seatContainer = document.getElementById("seats");
const priceDisplay = document.querySelector("h1");

let totalAmount = 0;
const seatPrice = 250;

for (let row = 1; row <= 3; row++) {
  for (let col = 1; col <= 3; col++) {

    const seatBtn = document.createElement("button");
```

```
seatBtn.textContent = `Row ${row}, Col ${col}`;

seatBtn.addEventListener("click", function () {

  if (seatBtn.classList.contains("booked")) {
    return;
  }

  seatBtn.classList.add("booked");

  totalAmount += seatPrice;
  priceDisplay.textContent = `Total Amount: ₹${totalAmount}`;
});

seatContainer.appendChild(seatBtn);
}

seatContainer.appendChild(document.createElement("br"));
}
```

OUTPUTS:-

 Your Work - CodePen  Js (forked) - StackBlitz  Gemini  JavaScript Playgrou...

Total: ₹0

Row 1, Col 1	Row 1, Col 2	Row 1, Col 3
Row 2, Col 1	Row 2, Col 2	Row 2, Col 3
Row 3, Col 1	Row 3, Col 2	Row 3, Col 3



Your Work - CodePen



Js (forked) - StackBlitz



Gemini



JavaScript Playgrou...

Total: ₹500

Row 1, Col 1	Row 1, Col 2	Row 1, Col 3
Row 2, Col 1	Row 2, Col 2	Row 2, Col 3
Row 3, Col 1	Row 3, Col 2	Row 3, Col 3

- Ex 6: Design an entry system for students attending an entrance exam. 10 students are attending the exam. The register numbers of the students are 101, 102..., 110.
- Develop a jQuery code such that when the student enters the gate the exam supervisor can enter the register number of the candidate in a textbox and click the "verify" button. Perform appropriate event-handling such that the register number is valid (with-in range of 101 to 110 inclusive). There must be no repetition of the same register number already entered the hall after gate check.
 - If the entered register number is correct, then display it in a dynamically generated queue with register number displayed in a `<h3>` as text. If there is an invalid entry then display as "Check other venue" in the queue. Perform this operation using jQuery and queue system using DOM.

Index.html:-

```
<!DOCTYPE html>
<html>
<head>
  <title>Exam Entry System</title>

  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

  <style>
    body {
      font-family: Arial, sans-serif;
      background: linear-gradient(to right, #667eea, #764ba2);
      text-align: center;
```



```
    margin: 0;
    padding: 0;
}

.container {
    background: white;
    width: 350px;
    margin: 80px auto;
    padding: 25px;
}

h2 {
    margin-bottom: 20px;
}

input {
    padding: 10px;
    width: 70%;
    border: 1px solid #ccc;
    border-radius: 5px;
}

button {
    padding: 10px 15px;
    background: #4CAF50;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    margin-left: 5px;
}

button:hover {
    background: #45a049;
}

#queue {
    margin-top: 20px;
    text-align: left;
}

.entry {
    padding: 8px;
    margin: 5px 0;
    border-radius: 5px;
}

.valid {
```

```

        background: #e6ffe6;
        color: green;
        border-left: 5px solid green;
    }

    .invalid {
        background: #ffe6e6;
        color: red;
        border-left: 5px solid red;
    }
</style>
</head>

<body>

    <div class="container">
        <h2>Exam Entry System</h2>

        <input type="number" id="regNo" placeholder="Enter Reg No (101-110)">
        <button id="verify">Verify</button>

        <div id="queue"></div>
    </div>

    <script>
        $(document).ready(function () {

            let entered = [];

            $("#verify").click(function () {

                let reg = parseInt($("#regNo").val());
                let element;

                // Validation
                if (reg >= 101 && reg <= 110) {

                    if (!entered.includes(reg)) {
                        entered.push(reg);

                        element = $("<div class='entry valid'></div>")
                            .text("Reg No: " + reg);
                    } else {
                        element = $("<div class='entry invalid'></div>")
                            .text(" Check other venue (Duplicate)");
                    }

                } else {

```

```
        element = $("<div class='entry invalid'></div>")
            .text("Check other venue (Invalid)");
    }
    $("#queue").append(element);
    $("#regNo").val("");
});

});
</script>

</body>
</html>
```

OUTPUTS:-

The screenshot displays a web application titled "Exam Entry System". It features a search bar with the placeholder text "Enter Reg No (101–110)" and a green "Verify" button. Below the search bar, there is a list of registered numbers, each displayed in a green box with a green vertical bar on the left: "Reg No: 101", "Reg No: 102", "Reg No: 103", and "Reg No: 108". At the bottom of the list, there is a red box with a red vertical bar on the left, containing the text "Check other venue (Invalid)".

Ex 7:	Design a web-based monitoring application using Node.js to evaluate the performance of a newly developed drone obstacle avoidance system. The application must fulfill the following requirements: • Display real-time flight parameters such as latitude and longitude on an HTML page. • Expose an HTTP endpoint that returns the type of obstacle detected during flight in JSON format. The obstacle types may include: "bird", "structure", or "tree". NOTE: Assume in the current instance the obstacle is "Tree", any latitude and longitude.
-------	--

Server.js:-

```
const express = require("express");
const app = express();

app.use(express.static("public"));

app.get("/obstacle", (req, res) => {
  res.json({
    obstacle: "tree"
  });
});

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

Index.html:-

```
<!DOCTYPE html>
<html>
<head>
  <title>Drone Monitor</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">

  <h2>Drone Monitoring System</h2>

  <h3>Latitude: <span id="lat"></span></h3>
  <h3>Longitude: <span id="lon"></span></h3>

  <button onclick="getObstacle()">Check Obstacle</button>
```

```
<h3 id="result"></h3>

</div>

<script>
  setInterval(() => {
    document.getElementById("lat").innerText = (Math.random() * 100).toFixed(2);
    document.getElementById("lon").innerText = (Math.random() * 100).toFixed(2);
  }, 1000);

  function getObstacle() {
    fetch("/obstacle")
      .then(res => res.json())
      .then(data => {
        document.getElementById("result").innerText =
          "Obstacle Detected: " + data.obstacle;
      });
  }
</script>

</body>
</html>
```

Style.css:-

```
body {
  font-family: Arial;
  background: linear-gradient(to right, #0f2027, #203a43, #2c5364);
  color: white;
  text-align: center;
}

.container {
  margin-top: 100px;
}

button {
  padding: 10px 20px;
  background: #ff9800;
  border: none;
  color: white;
  border-radius: 5px;
  cursor: pointer;
}

button:hover {
  background: #e68900;
}
```

OUTPUTS:-

Ex 8:	Design a board for displaying cricket statistics using MERN stack in a single route. • Display the name of the cricket player with leading runs. • Display the names of the cricket players in descending order of wickets taken. • Display the names of any two cricket players who scored century. • Display the names of cricket players who scored runs greater than 50. • Display the count of cricket players who scored century. NOTE: Assume the collection has names of cricket players, runs and wickets_taken as keys.
-------	---

Server.js:-

```

const express = require('express');
const mongoose = require('mongoose');

const app = express();
const PORT = 3000;

mongoose.connect("mongodb://localhost:27017/cricketDB")
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log(err));

const playerSchema = new mongoose.Schema({
  name: String,
  runs: Number,
  wickets_taken: Number
});

```

```

const Player = mongoose.model("Player", playerSchema);

async function insertSampleData() {

  const count = await Player.countDocuments();

  if (count === 0) {
    await Player.insertMany([
      { name: "Virat", runs: 120, wickets_taken: 4 },
      { name: "Rohit", runs: 80, wickets_taken: 1 },
      { name: "Hardik", runs: 60, wickets_taken: 5 },
      { name: "Bumrah", runs: 20, wickets_taken: 6 },
      { name: "Gill", runs: 150, wickets_taken: 0 }
    ]);
    console.log("Sample Data Inserted");
  }
}

insertSampleData();

app.get("/", async (req, res) => {

  const highestRunPlayer = await Player.findOne().sort({ runs: -1 });

  const wicketsDescending = await Player.find().sort({ wickets_taken: -1 });

  const centuryPlayers = await Player.find({ runs: { $gte: 100 } }).limit(2);

  const runsGreaterThan50 = await Player.find({ runs: { $gt: 50 } });

  const centuryCount = await Player.countDocuments({ runs: { $gte: 100 } });

  res.send(`
    <h2>Cricket Statistics Board</h2>

    <h3>Player with Highest Runs:</h3>
    <p>${highestRunPlayer.name} (${highestRunPlayer.runs} runs)</p>

    <h3>Players Sorted by Wickets (Descending):</h3>
    <ul>
      ${wicketsDescending.map(p => `<li>${p.name} - ${p.wickets_taken} wickets</li>`).join("")}
    </ul>

    <h3>Two Century Players:</h3>
    <ul>
      ${centuryPlayers.map(p => `<li>${p.name}</li>`).join("")}
    </ul>

    <h3>Players with Runs > 50:</h3>
    <ul>
  `);
}

```

```

    ${runsGreaterThan50.map(p => `<li>${p.name}</li>`).join("")}
  </ul>

  <h3>Count of Century Players:</h3>
  <p>${centuryCount}</p>
  `);
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});

```

OUTPUTS:-

Cricket Statistics Board

Player with Highest Runs:

Gill (150 runs)

Players Sorted by Wickets (Descending):

- Bumrah - 6 wickets
- Hardik - 5 wickets
- Virat - 4 wickets
- Rohit - 1 wickets
- Gill - 0 wickets

Two Century Players:

- Virat
- Gill

Players with Runs > 50:

- Virat
- Rohit
- Hardik
- Gill

Count of Century Players:

2

Ex 9: Develop a web-based application using MERN stack to facilitate paper submissions for a technical conference. The application must meet the following requirements:

- Paper Submission Interface
- Create an insertion page with a form that includes the following input fields:
 - Author Name
 - Co-author Name
 - Paper Title
 - Abstract
- Include a Submit button that, to insert the form data to the backend via the route: `http://localhost:3000/insert_data`
- Paper Title Update Interface

- Create an updating page with a form for editing paper title (only). Include input fields for entering paper title and its object ID. Include an Edit button to update the title in backend via the route: `http://localhost:3000/update_data`

Server.js:-

```
const express = require("express");
const mongoose = require("mongoose");

const bodyParser = require("body-parser");

const app = express();

app.use(bodyParser.urlencoded({ extended: true }));

app.use(express.static("public"));

mongoose.connect("mongodb://localhost:27017/conferenceDB")

.then(() => console.log("MongoDB Connected"))

.catch(err => console.log(err));

// Schema
const paperSchema = new mongoose.Schema({
  authorName: String,

  coAuthorName: String,

  paperTitle: String,

  abstract: String
});

const Paper = mongoose.model("Paper", paperSchema);

app.get("/", (req, res) => {
  res.redirect("/insert.html");
});

// ----- INSERT -----
app.post("/insert_data", async (req, res) => {

  const paper = new Paper(req.body);

  await paper.save();

  res.send("<h3>Paper submitted successfully</h3><a href='/insert.html'>Back</a>");
});

// ----- UPDATE -----
```

```

app.post("/update_data", async (req, res) => {

  const { id, paperTitle } = req.body;

  await Paper.findByIdAndUpdate(id, { paperTitle });

  res.send("<h3>Title updated successfully</h3><a href='/update.html'>Back</a>");
});

// ----- SERVER -----
app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});

```

Insert.html:-

```

<!DOCTYPE html>
<html>
<head>
  <title>Paper Submission</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="box">
  <h2>Submit Paper</h2>

  <form action="/insert_data" method="POST">

    <input name="authorName" placeholder="Author Name" required><br>

    <input name="coAuthorName" placeholder="Co-author Name" required><br>

    <input name="paperTitle" placeholder="Paper Title" required><br>

    <textarea name="abstract" placeholder="Abstract"></textarea><br>

    <button>Submit</button>
  </form>

</div>

</body>
</html>

```

Update.html:-

```

<!DOCTYPE html>
<html>
<head>
  <title>Update Paper</title>
  <link rel="stylesheet" href="style.css">
</head>

```

```
<body>

<div class="box">
  <h2>Update Paper Title</h2>

  <form action="/update_data" method="POST">

    <input name="id" placeholder="Paper ID" required><br>
    <input name="paperTitle" placeholder="New Title" required><br>

    <button>Edit</button>
  </form>

</div>

</body>
</html>
```

Style.css:-

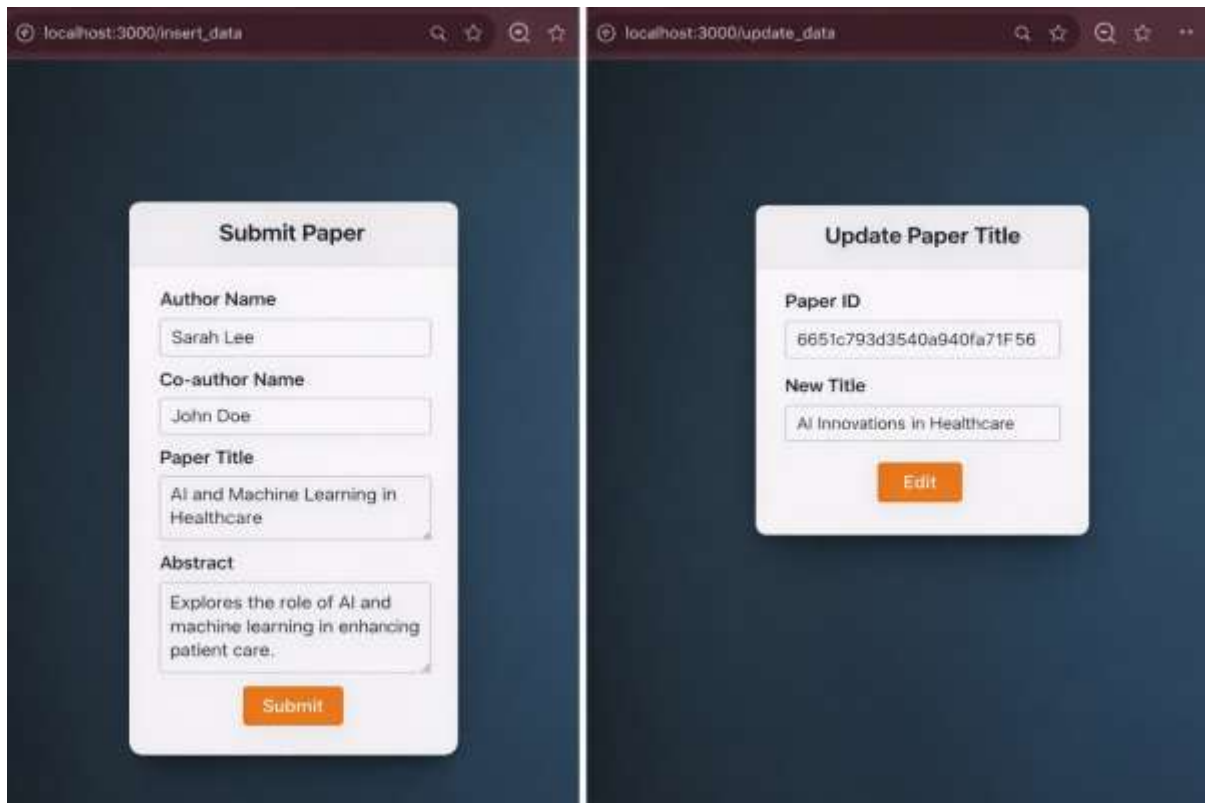
```
body {
  font-family: Arial;
  background: linear-gradient(to right, #232526, #414345);
  color: white;
}

.box {
  width: 300px;
  margin: 100px auto;
  padding: 20px;
  background: white;
  color: black;
  border-radius: 10px;
  text-align: center;
}

input, textarea {
  width: 90%;
  padding: 8px;
  margin: 10px;
}

button {
  padding: 8px 15px;
  background: #ff9800;
  border: none;
  color: white;
  cursor: pointer;
}
```

OUTPUTS:-



- Ex 10 Develop an e-newspaper using React based Single Page Application (SPA). The web page must be designed as follows:
- Create a stateless functional component called (NEWS) that displays the headline (in `<h1>`) tag, followed by the relevant image and the name of the NEWS reporter (in `<p>`) tag. Use appropriate props to set headlines, images and NEWS reporter names.
 - Create a stateful class component called (Editor) that displays the editor's e-mail displayed in a `<p>` tag.
 - One NEWS article has a sequence one NEWS component and one Editor component. <Sequence of article content: headline, image, reporter name and editor's e-mail>
 - Assume that on a day there are three NEWS articles. Create the e-news paper for that day by adding the child components appropriately to the parent component for the three articles. The editor's e-mail is the same for all the three articles.

App.js:-

```
import NEWS from './NEWS';
import Editor from './Editor';

import cricketImg from './cricket.jpg';
import techImg from './tech.jpg';
import stockImg from './stock.jpg';

function App() {
```

```

const editorEmail = "editor@news.com";

return (
  <div>

    {/* Article 1 */}
    <NEWS
      headline="India Wins Test Match"
      image={cricketImg}
      reporter="Rahul"
    />
    <Editor email={editorEmail} />

    <hr />

    {/* Article 2 */}
    <NEWS
      headline="Tech Conference 2026"
      image={techImg}
      reporter="Anita"
    />
    <Editor email={editorEmail} />

    <hr />

    {/* Article 3 */}
    <NEWS
      headline="Stock Market Hits High"
      image={stockImg}
      reporter="Suresh"
    />
    <Editor email={editorEmail} />

  </div>
);
}

export default App;

```

editor.js:-

```

import React, { Component } from 'react';

class Editor extends Component {
  render() {
    return (
      <p>Editor Email: {this.props.email}</p>
    );
  }
}

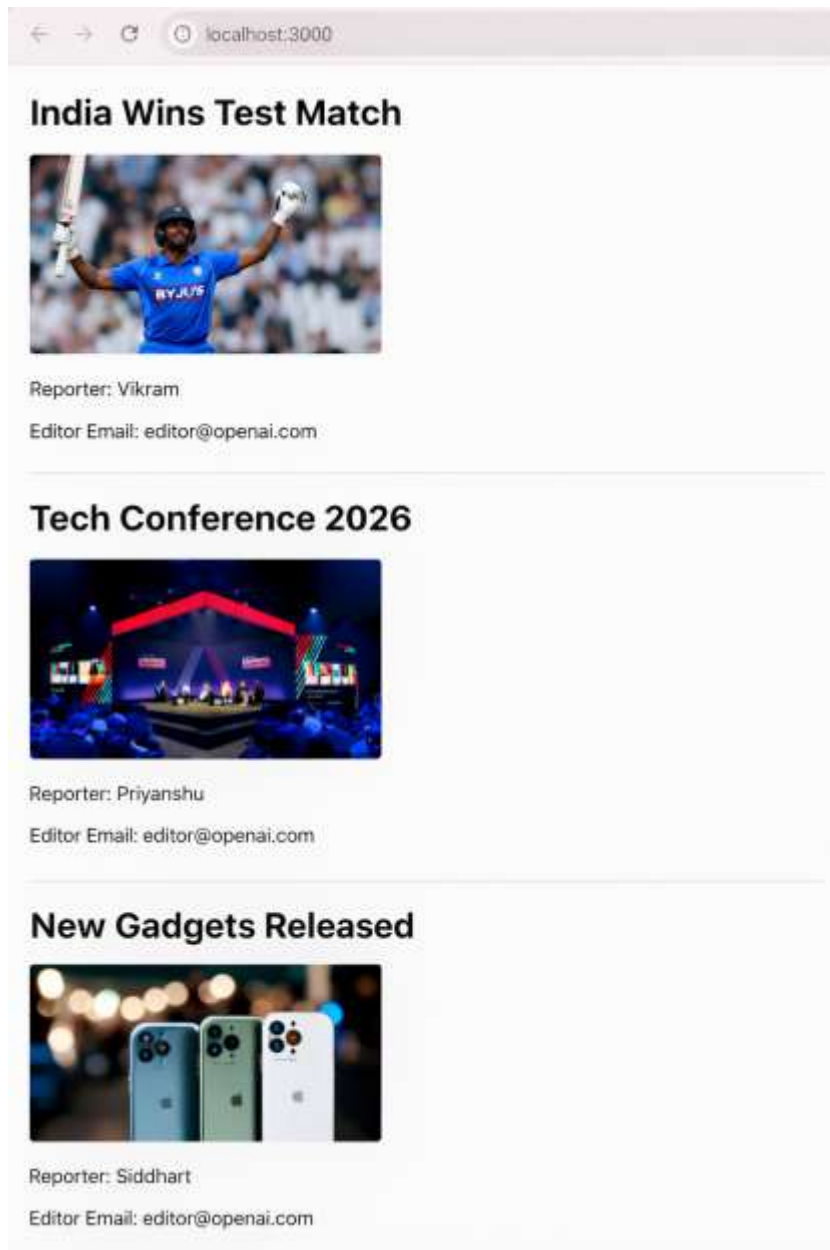
```

```
    );  
  }  
}  
  
export default Editor;
```

news.js:-

```
function NEWS(props) {  
  return (  
    <div>  
      <h1>{props.headline}</h1>  
      <img src={props.image} alt="news" width="300" />  
      <p>Reporter: {props.reporter}</p>  
    </div>  
  );  
}  
  
export default NEWS;
```

OUTPUTS:-



Ex 11:	A stock analyst was studying about the stock price variations of a particular company. Design a React based Single Page Application (SPA) that helps the stock analyst to study the dynamics. The SPA must have the following: - Using a stateful class component display the stock details of the company like symbol (ABC Technology), price per stock Rs. 300 and volume sold is 5000 as text in <code><h3></code> tag. This information will be displayed during mounting. - After 5 hours replace the initial price per stock and volume sold by the current data. The updating must be carried on the DOM tree. Develop the application using appropriate lifecycle methods.
--------	---

App.js:-

```
import React, { Component } from "react";
```

```

import './App.css';

class App extends Component {

  constructor() {
    super();

    this.state = {
      symbol: "ABC Technology",
      price: 300,
      volume: 5000
    };
  }

  componentDidMount() {

    console.log("Component Mounted");

    setTimeout(() => {

      this.setState({
        price: 350,
        volume: 7000
      });

    }, 5000);
  }

  render() {
    return (
      <div className="container">

        <h2>Stock Analysis Dashboard</h2>

        <h3>Company: {this.state.symbol}</h3>
        <h3>Price: ₹{this.state.price}</h3>
        <h3>Volume: {this.state.volume}</h3>

      </div>
    );
  }
}

export default App;

```

app.css:-

```

body {
  font-family: Arial;
  background: linear-gradient(to right, #000428, #004e92);
  color: white;
  text-align: center;
}

```



```

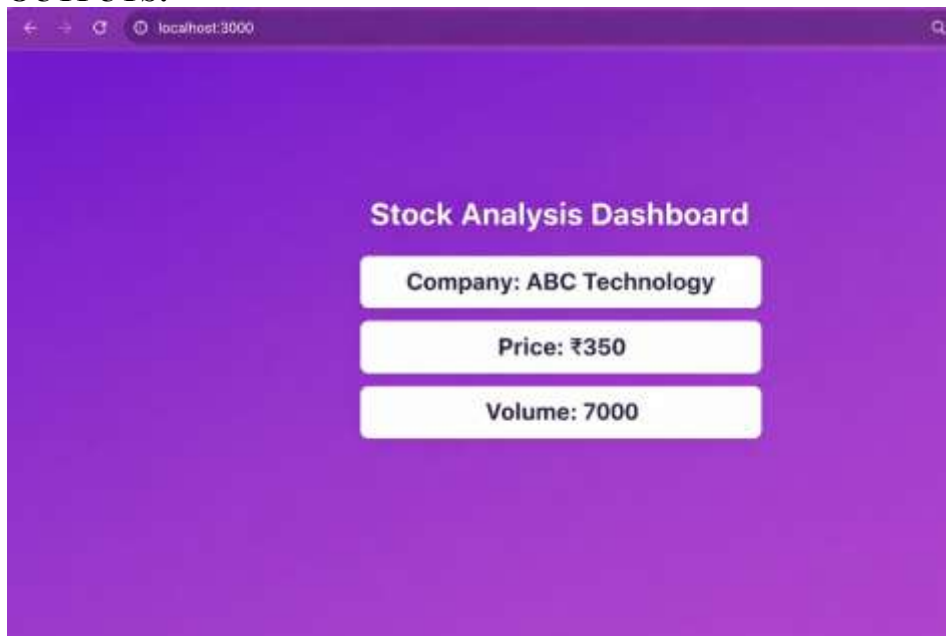
}

.container {
  margin-top: 100px;
}

h3 {
  background: white;
  color: black;
  width: 300px;
  margin: 10px auto;
  padding: 10px;
  border-radius: 8px;
}

```

OUTPUTS:-



Ex 12	The research division of a car manufacturing company wants you to build an application using node js as follows: - Develop a module in Common JS format called new_model. The new_model is a functional module that displays the specifications of a newly released car. Consume the function in an application and display the specifications. - Log the specification by writing to a "new_model.txt" file using appropriate pre-define module.
-------	---

New_model.js:-

```

function getCarSpecs() {
  return `

```

```

Car Model: XZ Turbo
Engine: 2.0L Petrol
Mileage: 18 km/l
Price: ₹15,00,000
};
}

```

```
module.exports = getCarSpecs;
```

app.js:-

```

const getCarSpecs = require("./new_model");
const fs = require("fs");

const specs = getCarSpecs();

console.log("Car Specifications:");
console.log(specs);

fs.writeFileSync("new_model.txt", specs);

console.log("Specifications saved to new_model.txt");

```

OUTPUTS:-

```

Car Specifications:

Car Model: ZX Electric
Engine: EV Motor (150 kW)
Mileage: 350 km/charge
Price: ₹22,50,000

Specifications saved to new_model.txt

```

- Ex 13 Assume that in shop owner wanted to monitor the functioning of his employees from an online application remotely. As a full stack application developer perform the following using node js:
- When the security opens the main gate at 7am morning a "security_open" event is triggered. When this event is triggered the owner must get a text that displays "Security on time". If there is a mismatch in time other than 7am then display as "Security out of time".
 - After the security opens then the employee in-charge of billing counter starts the billing software. This triggers a "billing_started" event and the owner must get a text that displays "Billing started at <Time>". <Time> is the when the billing software was started. Add another event listener to the "billing_started" event such that it displays the price of first and second bills to the owner.
- Write a code with appropriate event emitting sequence, parameters and event handling.

App.js:-

```
const EventEmitter = require("events");
```

```

const emitter = new EventEmitter();

emitter.on("security_open", (time) => {

  if (time === "7:00 AM") {
    console.log("Security on time");
  } else {
    console.log("Security out of time");
  }
});

emitter.on("billing_started", (time, bill1, bill2) => {

  console.log("Billing started at " + time);

});

emitter.on("billing_started", (time, bill1, bill2) => {

  console.log("First bill price: ₹" + bill1);
  console.log("Second bill price: ₹" + bill2);

});

emitter.emit("security_open", "7:00 AM");

emitter.emit("billing_started", "9:30 AM", 250, 400);

```

OUTPUTS:-

```

C:\Users\PriyanshuRathore\Desktop\node-project> node app.js

Security on time
Billing started at 9:30 AM
First bill price: ₹250
Second bill price: ₹400

C:\Users\PriyanshuRathore\Desktop\node-project>

```

Ex 14	Use AJAX and PHP to perform addition of two numbers without reloading the entire web page.
-------	--

Index.html:-

```

<!DOCTYPE html>
<html>

```

```

<head>
  <title>AJAX Addition</title>
</head>
<body>

<h2>Add Two Numbers</h2>

<input type="number" id="num1" placeholder="Enter first number">
<input type="number" id="num2" placeholder="Enter second number">

<button onclick="addNumbers()">Add</button>

<h3 id="result"></h3>

<script>
function addNumbers() {

  var n1 = document.getElementById("num1").value;
  var n2 = document.getElementById("num2").value;

  var xhr = new XMLHttpRequest();

  xhr.open("GET", "add.php?num1=" + n1 + "&num2=" + n2, true);

  xhr.onload = function() {
    if (this.status === 200) {
      document.getElementById("result").innerHTML =
        "Result: " + this.responseText;
    }
  };

  xhr.send();
}
</script>

</body>
</html>

```

Add.php:-

```

<?php

$num1 = $_GET['num1'];
$num2 = $_GET['num2'];

$sum = $num1 + $num2;

echo $sum;

?>

```

OUTPUTS:-**Add Two Numbers**

Add Two Numbers

Result:6

Ex 15 Design a web application using node js and express to check if the credentials (user name and password are entered correctly. If not the home page displays an error “Invalid credentials” If the credentials are correct then profile page is loaded and a session is created. The profile page displays your name, age, educational details and existing mobile number. The profile page has a button called “mark”. When “mark” is clicked, it redirects to another page that fetched your marks in two subjects and a submit button. On submission, the profile and total marks must be sent as e-mail to your parent. The page also has a signout button which on clicking redirects to home page and the session is terminated.

Server.js:-

```
const express = require("express");
const session = require("express-session");
const bodyParser = require("body-parser");
const nodemailer = require("nodemailer");

const app = express();

app.use(bodyParser.urlencoded({ extended: true }));
app.use(express.static("public"));

app.use(session({
  secret: "secret",
  resave: false,
  saveUninitialized: true
}));
```

```

const USER = {
  username: "admin",
  password: "1234",
  name: "Priyangshu",
  age: 22,
  education: "MCA",
  mobile: "9564418238",
  parentEmail: "pp@gmail.com"
};

app.get("/", (req, res) => {
  res.send(`
<link rel="stylesheet" href="/style.css">
<div class="box">
  <h2>Login</h2>
  <form method="POST" action="/login">
    <input name="username" placeholder="Username"><br>
    <input type="password" name="password" placeholder="Password"><br>
    <button>Login</button>
  </form>
</div>
`);
});

app.post("/login", (req, res) => {

  const { username, password } = req.body;

  if (username === USER.username && password === USER.password) {

    req.session.user = USER;
    res.redirect("/profile");

  } else {

    res.send(`
<h3>Invalid credentials</h3>
<a href="/">Back</a>
`);
  }
});

app.get("/profile", (req, res) => {

  if (!req.session.user) return res.redirect("/");

  res.send(`
<link rel="stylesheet" href="/style.css">
<div class="box">
  <h2>Profile</h2>

```

```

    <p>Name: ${USER.name}</p>
    <p>Age: ${USER.age}</p>
    <p>Education: ${USER.education}</p>
    <p>Mobile: ${USER.mobile}</p>

    <a href="/marks">Mark</a><br><br>
    <a href="/logout">Signout</a>
  </div>
  `);
});

app.get("/marks", (req, res) => {

  res.send(`
    <link rel="stylesheet" href="/style.css">
    <div class="box">
      <h3>Enter Marks</h3>

      <form method="POST" action="/send">
        <input name="m1" placeholder="Subject 1"><br>
        <input name="m2" placeholder="Subject 2"><br>
        <button>Submit</button>
      </form>
    </div>
    `);
});

app.post("/send", (req, res) => {

  const { m1, m2 } = req.body;
  const total = parseInt(m1) + parseInt(m2);

  const transporter = nodemailer.createTransport({
    service: "gmail",
    auth: {
      user: "your_email@gmail.com",
      pass: "your_app_password"
    }
  });

  const mailOptions = {
    from: "your_email@gmail.com",
    to: USER.parentEmail,
    subject: "Student Report",
    text: `
      Name: ${USER.name}
      Age: ${USER.age}
      Education: ${USER.education}
      Mobile: ${USER.mobile}

      Marks:

```

```

    Subject1: ${m1}
    Subject2: ${m2}
    Total: ${total}
    ,
  };

  transporter.sendMail(mailOptions, (err, info) => {
    if (err) {
      console.log(err);
      res.send("Error sending mail");
    } else {
      res.send("Email sent successfully");
    }
  });
});

app.get("/logout", (req, res) => {
  req.session.destroy();
  res.redirect("/");
});

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});

```

Style.css:-

```

body {
  font-family: Arial;
  background: linear-gradient(to right, #1e3c72, #2a5298);
  color: white;
  text-align: center;
}

.box {
  width: 300px;
  margin: 100px auto;
  padding: 20px;
  background: white;
  color: black;
  border-radius: 10px;
}

input {
  padding: 8px;
  margin: 10px;
  width: 80%;
}

button {
  padding: 8px 15px;
  background: #ff9800;

```



```
border: none;  
color: white;  
}  
  
a {  
text-decoration: none;  
color: blue;  
}
```

OUTPUTS:-

The image displays two screenshots of a web application running on localhost:3000. The top screenshot shows a 'Login' form with a title 'Login', a text input field containing 'admin', a password input field with four asterisks, and an orange 'Login' button. The bottom screenshot shows a 'Profile' page with a title 'Profile' and the following details: Name: Priyanshu, Age: 20, Education: MCA, and Mobile: 9876543210. At the bottom of the profile card are two blue links: 'Mark' and 'Signout'.

Enter Marks

99

95

Submit

Email Sent Successfully

Name: Priyanshu

Marks: 99, 95

Total: 194

[Back to Profile](#)

- Ex 16 Create a marksheet SPA(Single page application) to display marksheet of two students of a class using React. 1. Create a stateless functional child component that displays as “MARKSHEET XXX name NNNN” as . It must then display the image of the student and his marks in 3 subjects using ordered list. Here XXX is regno of the students given as props (regno) and NNNN is the name of the student given as props (name). The file name of this component is (Marksheet.js) 2. Create a stateful class child component that displays the signature of the proctor as an image and proctor name as a props (name). The file name of this child component is (Signature.js) The index page of the SPA displays marksheets of all students of the class by placing marksheet component followed by signature component for each students. To test your application populate your index.html with appropriate components for two students. Parent component contains child components as follows: (students1-marksheet-signature, student2-marksheet-student)

App.js:-

```
import Marksheet from './Marksheet';
```

```

import Signature from './Signature';

import student1Img from './student1.jpg';
import student2Img from './student2.jpg';
import signatureImg from './signature.jpg';

function App() {

  return (
    <div>

      { /* Student 1 */ }
      <Marksheet
        regno="101"
        name="AAKASHI"
        image={student1Img}
        marks={[85, 90, 88]}
      />

      <Signature
        image={signatureImg}
        name="Dr. Sharma"
      />

      <hr />

      { /* Student 2 */ }
      <Marksheet
        regno="102"
        name="NEHA"
        image={student2Img}
        marks={[78, 82, 80]}
      />

      <Signature
        image={signatureImg}
        name="Dr. Sharma"
      />

    </div>
  );
}

export default App;

```

marksheet.js:-

```

function Marksheet(props) {
  return (
    <div>
      <h2>MARKSHEET {props.regno} Name {props.name}</h2>

```

```
<img src={props.image} alt="student" width="200" />

<ol>
  <li>Maths: {props.marks[0]}</li>
  <li>Science: {props.marks[1]}</li>
  <li>English: {props.marks[2]}</li>
</ol>
</div>
);
}

export default Marksheet;
```

signature.js:-

```
import React, { Component } from 'react';

class Signature extends Component {
  render() {
    return (
      <div>
        <img src={this.props.image} alt="signature" width="150" />
        <p>Proctor: {this.props.name}</p>
      </div>
    );
  }
}

export default Signature;
```

OUTPUTS:-

← → ↺ ⓘ localhost:3001

MARKSHEET 101 Name Akash



1. Maths: 92
2. Science: 80
3. English: 91



Proctor: Dr. Sharma

MARKSHEET 102 Name Neha



1. Maths: 74
2. Science: 78
3. English: 85